



[(주)휴고다이나믹스 & 세종대학교 Medical Robotics LAB.]

다족보행 로봇 자율보행 기술 개발

세종대학교 AI로봇학과 우현수 / 2026.07.28 미팅

2026.07.28

세종대학교 Medical Robotics LAB.



아이작심 설치 및 활용

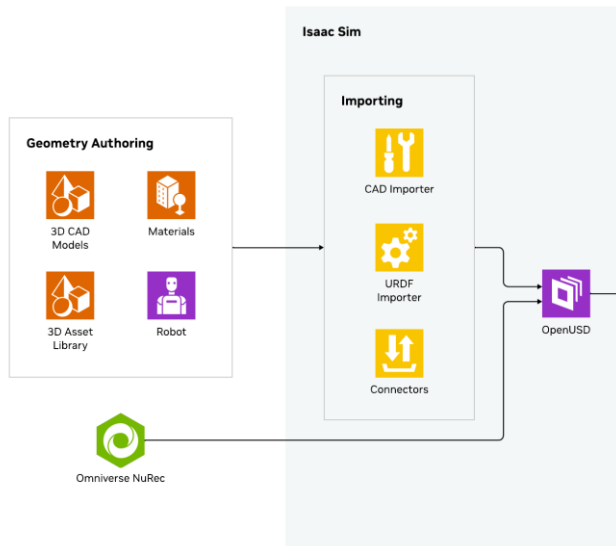


NVIDIA Isaac Sim

NVIDIA Omniverse™ 라이브러리 기반의 고성능 로보틱스 시뮬레이션 애플리케이션 및 합성 데이터 생성 도구

- 다중 GPU로 확장이 가능해 대규모 시뮬레이션도 빠르게 수행할 수 있음
- 물리 엔진 NVIDIA PhysX를 기반으로 강체 동역학, 다관절(Articulation), 차량 동역학, SDF 충돌체 등을 정밀하게 시뮬레이션
- 최근에는 Google DeepMind·Disney Research와 공동 개발한 차세대 오픈소스 물리 엔진 Newton도 지원함.

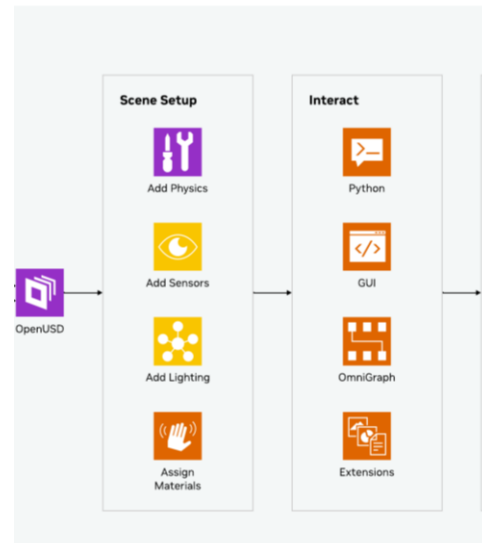
NVIDIA Isaac Sim



모델 импорт

CAD·URDF → USD 변환

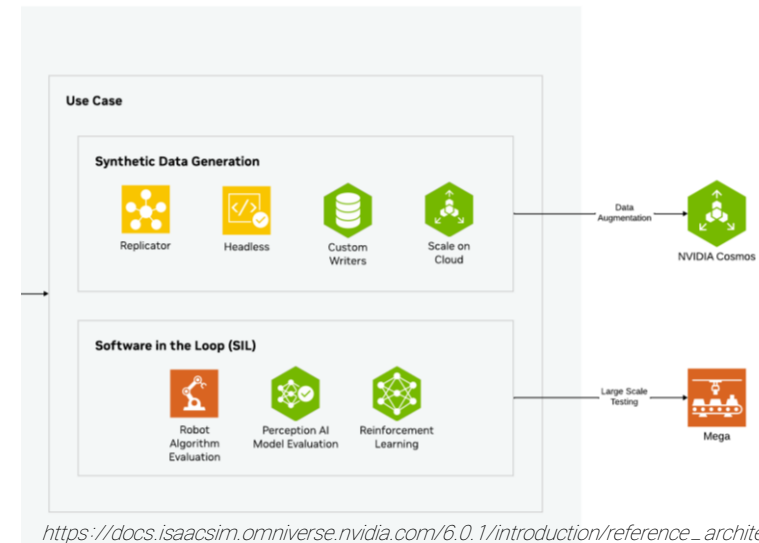
Isaac Sim은 CAD, URDF, MJCF, OnShape 등 다양한 형식의 로봇 모델을 импорт, USD 포맷으로 변환되어 시뮬레이션함



장면 구성

재질·물리 속성·센서 설정

카메라, LiDAR 등 가상 센서를 로봇에 부착해 실제와 유사한 센서 데이터를 얻을 수 있고, ROS/ROS2 브릿지를 통해 실제 로봇에서 쓰는 소프트웨어 스택과 시뮬레이션을 직접 연동할 수도 있다.



https://docs.isaacsim.omniverse.nvidia.com/6.0.1/introduction/reference_architecture.html

검증 및 활용

End-to-End 검증

이렇게 구성된 환경 위에서 SIL(Software-in-the-Loop), HIL(Hardware-in-the-Loop) 방식으로 전체 시스템을 검증할 수 있어, 실물 로봇 제작 전 단계에서 개발·테스트 사이클을 크게 단축시킴

NVIDIA Isaac Sim

I 활용분야

합성 데이터 생성(Synthetic Data Generation)

조명·색상·물체 위치 등을 랜덤화해 AI 모델 학습에 필요한 라벨링된 데이터를 대량으로 생성



산업 시설 디지털 트윈

공장이, 물류창고 같은 시설을 가상으로 구축해 다중 로봇 플릿 운영 시뮬레이션



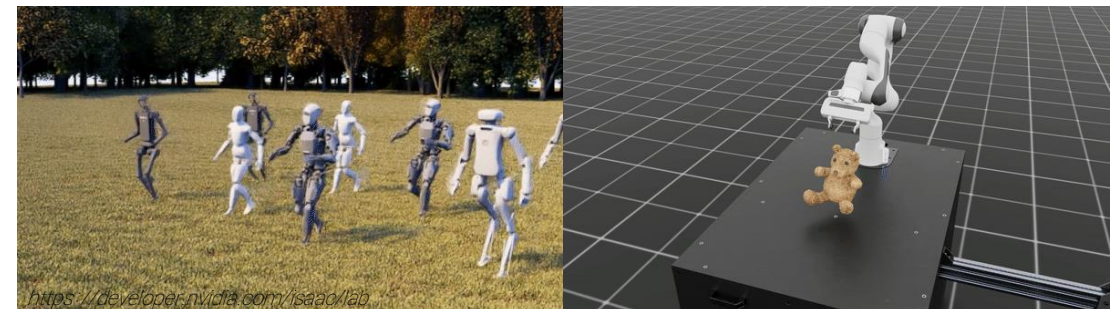
실세계 재구성(NuRec, Neural Reconstruction)

실제 센서 데이터를 3D Gaussian Splatting 기반으로 렌더링해 현실 공간을 그대로 시뮬레이션 환경으로 이관



로봇 학습(Robot Learning)

Isaac Lab을 통해 강화학습 기반의 로봇 제어 정책을 학습

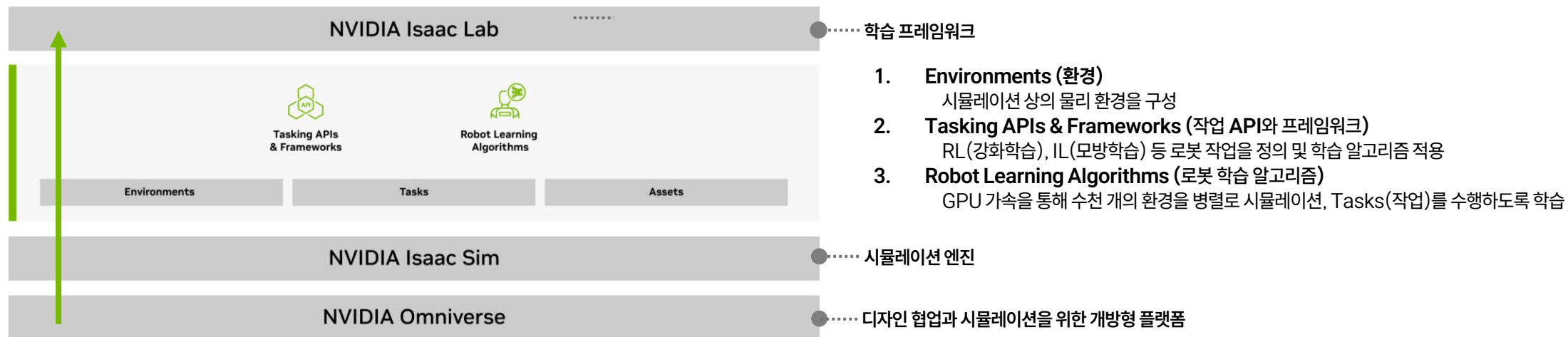


NVIDIA Isaac Sim

I Isaac Lab

로봇 정책을 대규모로 훈련하기 위해 설계된 로봇 학습용 오픈 소스, GPU 가속, 에이전트 지원 시뮬레이션 프레임워크

Nvidia 로봇틱스 워크플로우



<https://developer.nvidia.com/isaac/lab>



NVIDIA Isaac Sim

I 설치 방법

아이작 심을 설치하기 위해선 다음과 같은 환경이 필요함:

System Requirements			
Element	Minimum Spec	Good	Ideal
OS	Ubuntu 20.04/22.04 Windows 10/11	Ubuntu 20.04/22.04 Windows 10/11	Ubuntu 20.04/22.04 Windows 10/11
CPU	Intel Core i7 (7th Generation) AMD Ryzen 5	Intel Core i7 (9th Generation) AMD Ryzen 7	Intel Core i9, X-series or higher AMD Ryzen 9, Threadripper or higher
Cores	4	8	16
RAM	32GB ^[1]	64GB ^[1]	64GB ^[1]
Storage	50GB SSD	500GB SSD	1TB NVMe SSD
GPU	GeForce RTX 3070	GeForce RTX 4080	RTX Ada 6000
VRAM	8GB ^[1] ^[2]	16GB	48GB

<https://docs.isaacsim.omniverse.nvidia.com/latest/installation/requirements.html#isaac-sim-compatibility-checker>

현재 세종대 연구실 시스템 및 설치 환경 정보:

운영체제 (OS): Linux (Ubuntu 24.04 LTS / x86 _ 64)

Conda 가상환경 이름: env _ isaac lab

Python 버전: 3.11 (Isaac Sim 5.1.0 전용 필수 버전)

C/C++ 컴파일러: GCC 11 / G++ 11 (GCC 12, 13 미지원)

PyTorch / CUDA 버전: torch == 2.7.0 + cu128, torchvision == 0.22.0 + cu128

Isaac Sim 버전: 5.1.0

Isaac Lab 버전: 최신 (main 브랜치)

NVIDIA Isaac Sim

I 설치 방법



```
# 1. 패키지 목록 업데이트 및 기본 빌드 도구 설치
sudo apt-get update
sudo apt-get install build-essential -y

# 2. GCC 11 / G++ 11 설치
sudo apt-get install gcc-11 g++-11 -y

# 3. 심볼릭 링크(Alternatives) 설정 (기본 컴파일러를 11로 지정)
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 200
sudo update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-11 200

# 4. 버전 확인
gcc --version
g++ --version
```

최신 Ubuntu는 기본적으로 더 높은 버전(GCC 12/13)의 C/C++ 컴파일러를 제공한다. 하지만 Isaac Sim 5.1.0의 핵심 물리 엔진 모듈은 GCC 11 환경에서 빌드 및 최적화되어 있다.

시스템의 기본 컴파일러를 강제로 11 버전으로 낮춰(심볼릭 링크 설정), 이후 진행될 C++ 기반의 엔진 모듈 컴파일이 정상적으로 수행되도록 환경을 맞추는 가장 기초적인 공사이다.

1. 패키지 목록 업데이트 및 기본 빌드 도구 설치
2. GCC 11 / G++ 11 설치
3. 심볼릭 링크(Alternatives) 설정 (기본 컴파일러를 11로 지정)



NVIDIA Isaac Sim

I 설치 방법



```
# 1. Git 및 Git LFS 설치
sudo apt install git git-lfs -y

# 2. Isaac Sim 소스코드 다운로드 및 대용량 파일(LFS) 가져오기
git clone https://github.com/isaac-sim/IsaacSim.git isaacsim
cd isaacsim
git lfs install
git lfs pull

# 3. 빌드 프로세스 실행
./build.sh
```

GitHub에서 Isaac Sim의 원본 엔진을 가져오는 과정이다.

3D 물리 엔진의 특성상 로봇 모델, 텍스처 등 대용량 바이너리 파일이 많아, Git LFS(Large File Storage)를 연동하여 무거운 그래픽/물리 에셋 파일들을 안전하게 가져온 뒤, 스크립트를 통해 시스템에 맞게 엔진을 컴파일한다.

1. Git 및 Git LFS 설치
2. Isaac Sim 소스코드 다운로드 및 대용량 파일(LFS) 가져오기
3. 빌드 프로세스 실행

NVIDIA Isaac Sim

I 설치 방법



```
# 1. 홈 디렉토리로 이동하여 Miniconda 다운로드 및 설치 (미설치 시)
cd ~
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh

# 2. 터미널 환경 반영
source ~/.bashrc

# 3. 약관 동의
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r

# 4. Python 3.11 가상환경 생성 및 활성화
conda create -n env_isaialab python=3.11 -y
conda activate env_isaialab
gcc --version
g++ --version
```

시스템에 기본으로 설치된 파이썬 환경과 충돌하지 않도록, Isaac Lab 프로젝트만을 위한 완전히 독립적인 '가상 공간 (샌드박스)'을 구축하는 과정이다.

Isaac Sim 5.1.0은 반드시 파이썬 3.11 버전에서만 작동하도록 엄격하게 설계되어 있다. 따라서 Miniconda를 활용해 정확히 3.11 버전이 세팅된 전용 환경(env_isaialab)을 만들고 활성화하여, 앞으로 설치될 라이브러리들이 꼬이지 않고 안전하게 설치될 수 있다.

1. 홈 디렉토리로 이동하여 Miniconda 다운로드 및 설치 (미설치 시)
2. 터미널 환경 반영
3. 약관 동의
4. Python 3.11 가상환경 생성 및 활성화

NVIDIA Isaac Sim

I 설치 방법



```
# 1. pip 최신화
pip install --upgrade pip

# 2. Isaac Sim 5.1.0 파이썬 패키지 설치
pip install "isaacsim[all,extscache]==5.1.0" --extra-index-url https://pypi.nvidia.com

# 3. Isaac Sim 5.1.0 전용 PyTorch(cu128 기반) 설치
pip install torch==2.7.0 torchvision==0.22.0 --index-url https://download.pytorch.org/whl/cu128

# 4. SymPy 버전 충돌 교정 (isaacsim-core 요구사항 반영)
pip install sympy==1.13.3
```

앞서 만든 빈 가상환경에 3D 시뮬레이터 제어와 딥러닝 연산에 필요한 핵심 라이브러리들을 채워 넣는 단계이다.

NVIDIA의 Isaac Sim 전용 파이썬 패키지와, 이에 정확히 호환되는 GPU 연산용 PyTorch 2.7.0 버전을 차례대로 설치한다. 특히 이 과정에서 수학 라이브러리(SymPy)가 Isaac Sim 코어가 요구하는 버전과 어긋나는 의존성 충돌이 발생한다.

1. pip 최신화
2. Isaac Sim 5.1.0 파이썬 패키지 설치
3. Isaac Sim 5.1.0 전용 PyTorch(cu128 기반) 설치
4. SymPy 버전 충돌 교정 (isaacsim-core 요구사항 반영)

NVIDIA Isaac Sim

I 설치 방법



```
# 1. 홈 디렉토리로 이동
cd ~

# 2. 빌드 필수 도구 확인
sudo apt install cmake build-essential -y

# 3. Isaac Lab 소스코드 다운로드
git clone https://github.com/isaac-sim/IsaacLab.git

# 4. IsaacLab 디렉토리 이동
cd IsaacLab

# 5. 설치 스크립트 실행
./isacclab.sh --install
```

실제 로봇의 강화학습 및 시뮬레이션 환경 구성을 담당할 프레임워크를 설치하는 단계이다.

코드를 다운받은 후 설치 스크립트를 가동하면, 앞서 세팅한 목적한 Isaac Sim 코어 엔진과 파이썬 API들이 서로 통신할 수 있도록 연결(Binding)하는 작업이 내부적으로 진행된다. 스크립트 마지막의 약관(EULA) 동의는 NVIDIA의 GPU 가속 및 렌더링 코어 모듈을 사용하기 위한 필수 라이선스 승인 절차이다.

1. 빌드 필수 도구 확인
2. Isaac Lab 소스코드 다운로드
3. IsaacLab 디렉토리 이동
4. 설치 스크립트 실행

NVIDIA Isaac Sim

I 설치 방법



```
# (env_isaaclab 가상환경 및 ~/IsaacLab 디렉토리 내부에서 실행)
./isaacclab.sh -p scripts/tutorials/00_sim/create_empty.py

# 1단계: 가상환경 활성화
conda activate env_isaacclab

# 2단계: IsaacLab 폴더 이동
cd ~/IsaacLab

# 3단계: 파이썬 스크립트 실행
./isaacclab.sh -p [실행할_스크립트_경로.py]
```

실제 로봇의 강화학습 및 시뮬레이션 환경을 담당할 프레임워크를 설치한다.

코드를 다운받은 후 설치 스크립트를 가동하면, 앞서 세팅한 목적인 Isaac Sim 코어 엔진과 파이썬 API들이 서로 통신할 수 있도록 연결(Binding)하는 작업이 내부적으로 진행된다.

연산 부하가 가장 적은 '빈 환경(Empty Scene)' 스크립트를 실행하여, GPU를 활용한 초기 셰이더(Shader) 컴파일이 정상적으로 완료되고 그래픽 사용자 인터페이스(GUI) 창이 문제없이 로드되는지 눈으로 직접 확인한다.

아래 세 명령어는 앞으로 시스템을 구동할 때 사용한다. 터미널에서 파이썬 가상환경을 깨우고, 작업 디렉토리로 이동한 뒤, 전용 래퍼 스크립트(isaacclab.sh)를 통해서 파이썬 코드를 실행해야 한다.

NVIDIA Isaac Sim

I 설치 방법

필수 빌드 도구 및 GCC 11
다운그레이드

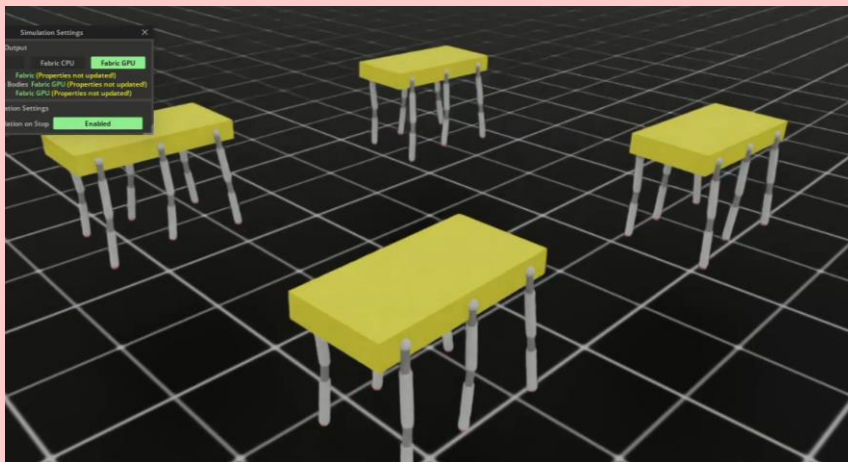
Isaac Sim 소스코드
다운로드 및 빌드

Conda 환경 구축

Isaac Sim &
PyTorch 패키지 설치

Isaac Lab 소스코드
다운로드 및 설치

설치 완료 검증



실제 로봇의 강화학습 및 시뮬레이션 환경을 담당할 프레임워크를 설치한다.

코드를 다운받은 후 설치 스크립트를 가동하면, 앞서 세팅한 목적인 Isaac Sim 코어 엔진과 파이썬 API들이 서로 통신할 수 있도록 연결(Binding)하는 작업이 내부적으로 진행된다.

연산 부하가 가장 적은 '빈 환경(Empty Scene)' 스크립트를 실행하여, GPU를 활용한 초기 셰이더(Shader) 컴파일이 정상적으로 완료되고 그래픽 사용자 인터페이스(GUI) 창이 문제없이 로드되는지 눈으로 직접 확인한다.

아래 세 명령어는 앞으로 시스템을 구동할 때 사용한다. 터미널에서 파이썬 가상환경을 깨우고, 작업 디렉토리로 이동한 뒤, 전용 래퍼 스크립트(isaacclab.sh)를 통해서 파이썬 코드를 실행해야 한다.

2026.07.28

세종대학교 Medical Robotics LAB.

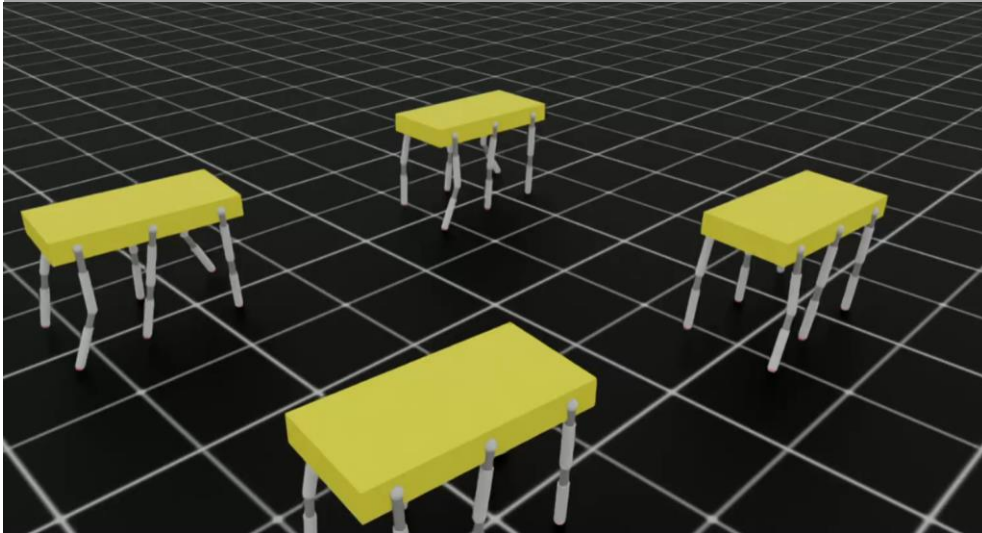


개선 사항 및 지표

개선 사항

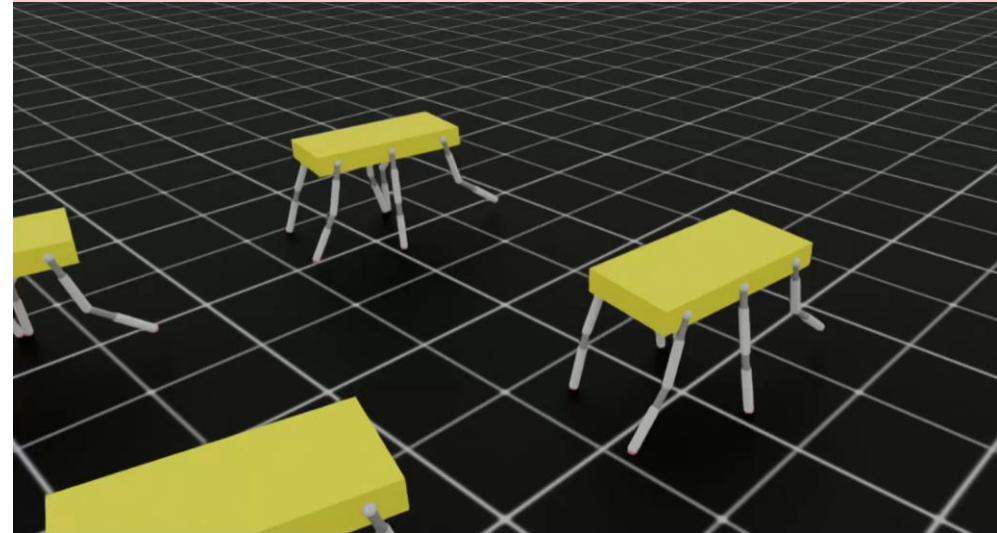
I Fluctuation 억제 - 보상 설계 개선 효과

이전 평지 보행



- 기본 보행 보상

개선 후 평지 보행



- 높이 유지 + 요동 억제 보상

동일 평지 조건, 동일 명령 속도(0.3 m/s)에서의 보행 비교 → 개선 후 몸통 상하 요동과 끄덕임이 육안으로 감소
lin_vel_z_l2 항 기준 물리량이 보상 강화 후 0.104 → 0.087로 (-16%), ang_vel_xy 기준 0.076 to 0.048 (-37%)

개선 사항

I Fluctuation 억제 - 보상 설계 개선 효과

평지 보행만 완료했을 때 요철 지면 보행



힘지 적응 학습 후 요철 지면 보행

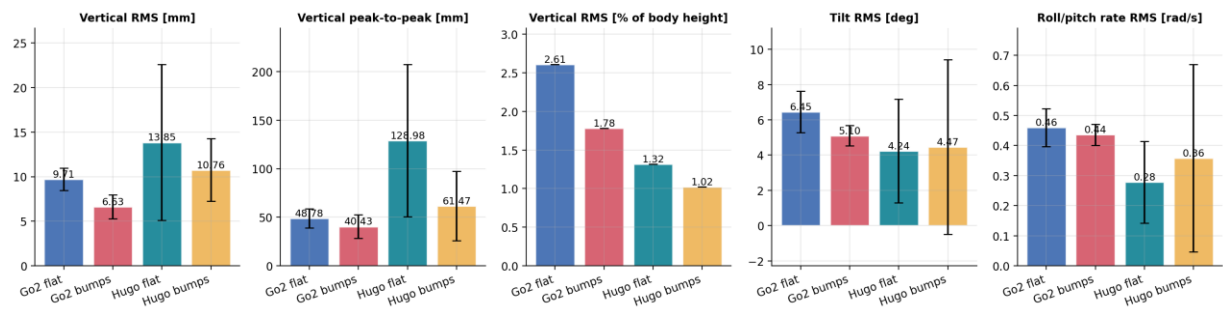


동일 $\pm 30\text{mm}$ 요철, 동일 명령 0.3 m/s — 평지 정책은 발이 요철에 걸려 감속·비틀거림, 적응 학습 후 정상 보행

	평지 정책	힘지 적응 정책
속도 달성률	62% (0.185)	104% (0.313)
보행 주파수	3.45 Hz (비정상)	0.88 Hz (정상 대역)
틸트 RMS	4.58°	4.47° (유지)

성능 지표

| 수직변동 RMS: 절대값과 체고 정규화

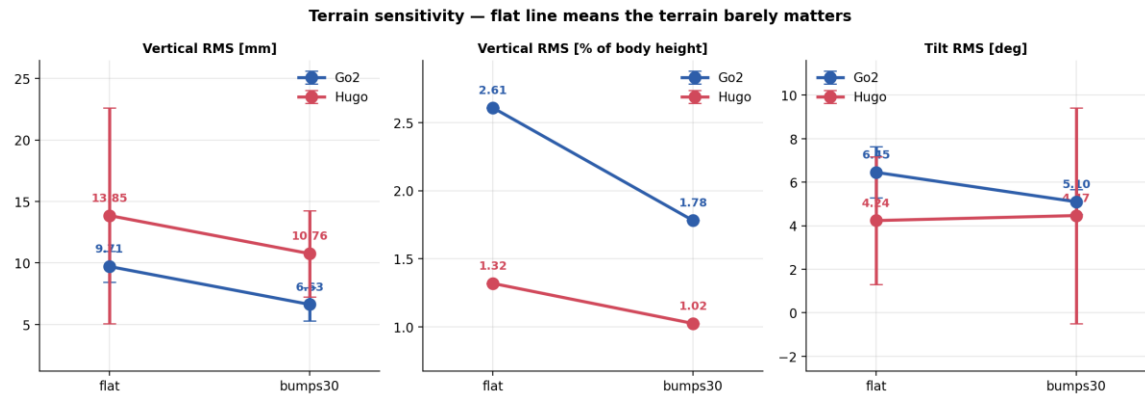


수직변동 RMS : 보행 중 몸통 높이가 평균에서 벗어난 정도의 제곱평균으로, 기계가 위아래로 흔들리는 평균 크기임

절대값(mm)은 체고 0.30m인 Go2에 구조적으로 유리하므로, 체고 대비 백분율로 정규화한 값이 서로 다른 크기의 로봇을 비교하는 공정한 척도임

절대값은 Go2가 앞서지만 체고 정규화 시 Hugo가 양 지형 모두 약 1.8배 우세

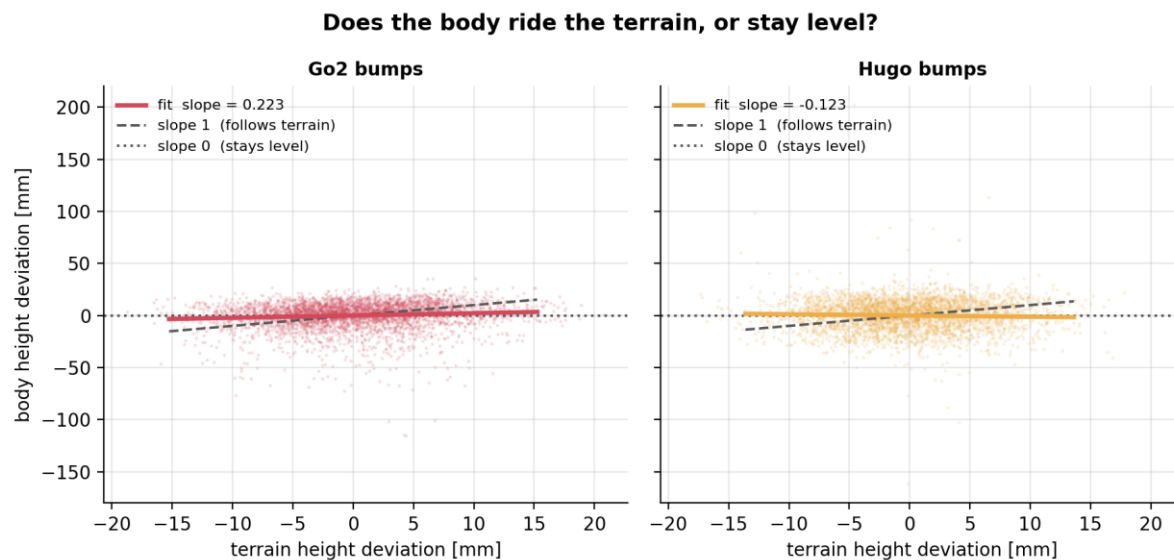
절대 mm는 체구가 작은 Go2가 유리하나, 자기 몸 크기 대비로는 Hugo가 크게 앞선다. 계획서 목표(요철 <15mm)도 10.76mm로 달성..



	Go2	Hugo
Bumps(험지) 절대 [mm]	6.63	10.76
Bumps(험지) 체고 [%]	1.78	1.02
Flat(평지) 체고 [%]	2.61	1.32

성능 지표

| 지형 추종 계수: 몸통이 지형을 타는가, 수평을 유지하는가



지형 추종 계수 : 지면이 1mm 오르내릴 때 몸통이 몇 mm 따라 움직이는지를 회귀 기울기로 측정한 값임

1이면 몸통이 지형을 그대로 타고 0이면 지형과 무관하게 수평을 유지함을 뜻함

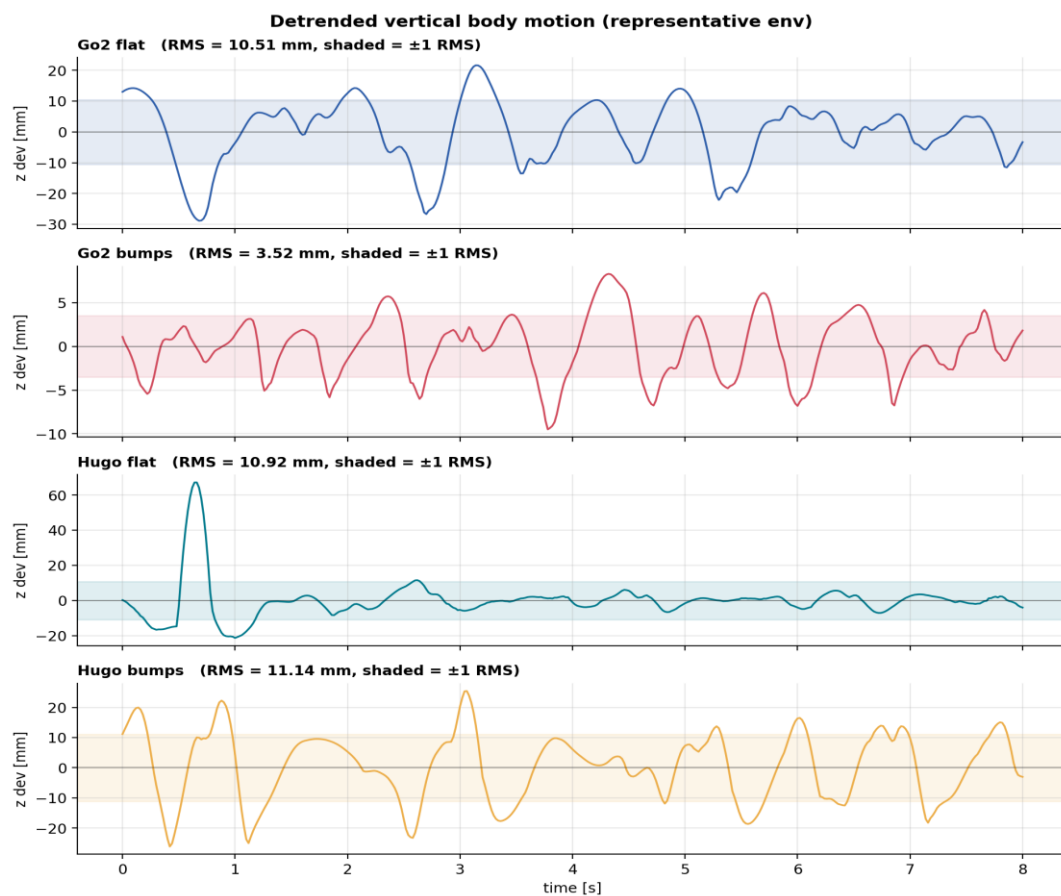
Go2는 0.223,

Hugo는 -0.123(≈ 0)으로 지면 굴곡을 다리 단계에서 흡수함

P-R 구조가 설계 목표(지형 외란의 다리 흡수)를 실제로 수행함을 보여주는 직접 증거. 이 한 장이 본 연구의 가설 검증에 해당한다.

성능 지표

I 이동 성능: 명령 속도 달성률



수직변동 RMS : 명령 속도 0.3m/s 대비 실제 평균 전진 속도임

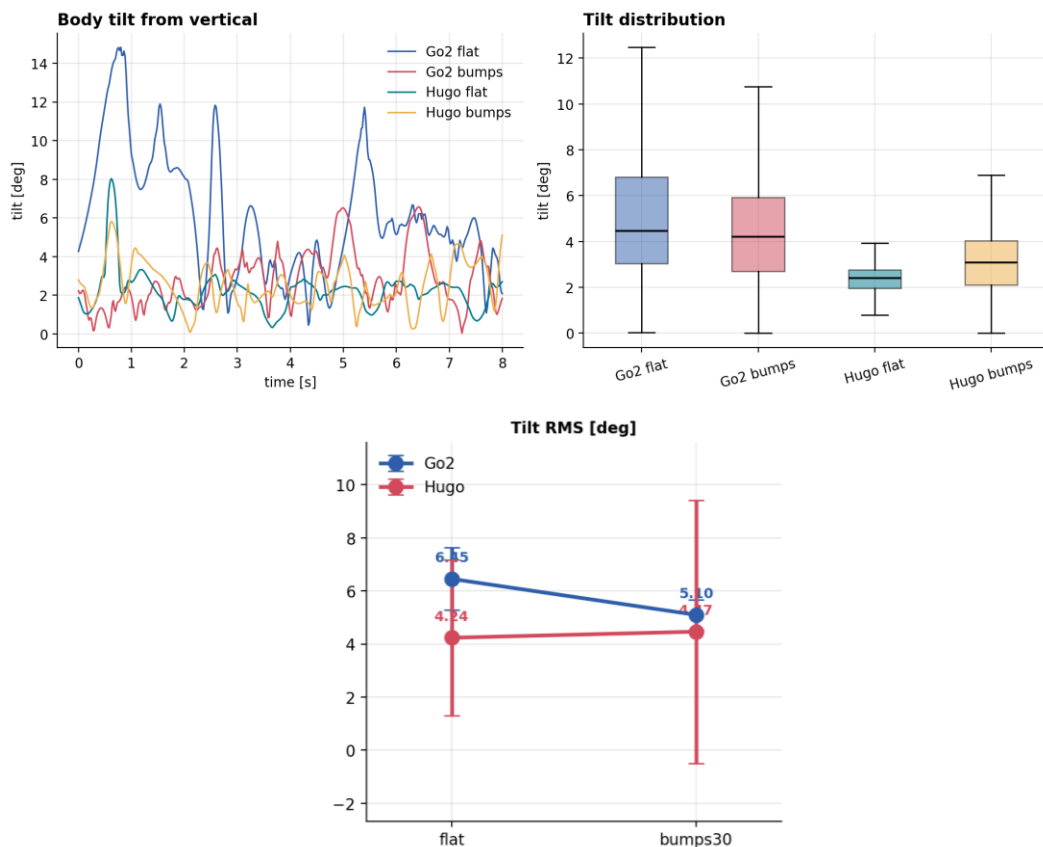
Hugo는 평지 91%(0.274), 요철 지형 104%(0.313)로 명령을 완주했으며, Go2는 요철 지형에서 83%(0.249)에 그쳤다.

요철 지형 전용 정책 학습으로 이전 측정의 속도 미달(62%) 문제를 해소했고, 이후의 모든 안정성 수치는 제 속도 보행 중의 값이다.

	명령 속도[m/s]	실제 속도[m/s]	달성률[%]
Go2 flat	0.30	0.293	97%
Go2 bumps	0.30 m/s	0.249	83%
Hugo flat	0.30 m/s	0.274	91%
Hugo bumps	0.30 m/s	0.313	104% (달성)

성능 지표

I 자세 안정성: 틸트각과 지형 민감도



틸트각: 몸통이 연직선에서 기울 각도

Tilt RMS: 평균적 기울

P95, 최댓값: 순간적으로 크게 기울는 정도를 나타냄

Hugo는 RMS 4.24°(flat)/4.47°(bumps)로
Go2(6.45°/5.10°)보다 전 조건 낮고, 최댓값에서도 9.2° vs 13.0°로 여유가 큼.

핵심은 값 자체보다 기울기이다. 지형이 평지에서 요철로 바뀌어도 Hugo의 틸트는
4.24→4.47°로 거의 변하지 않음(Tilt RMS의 수평선).

6족 정적 지지 + 수평 유지 제어가 지형 변화에 둔감한 자세를 만든다는 뜻으로, 탑재
장비(카메라·센서) 관점에서 직접적인 장점임.

성능 지표

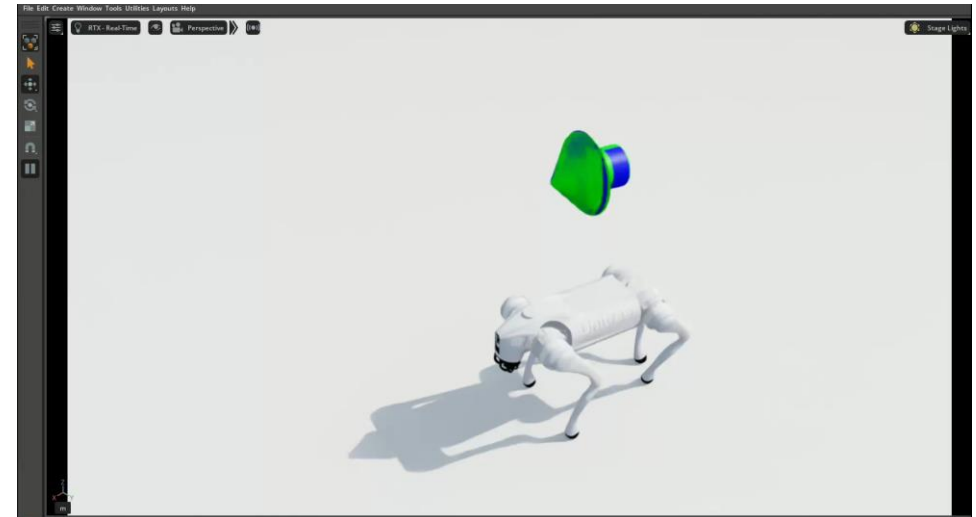
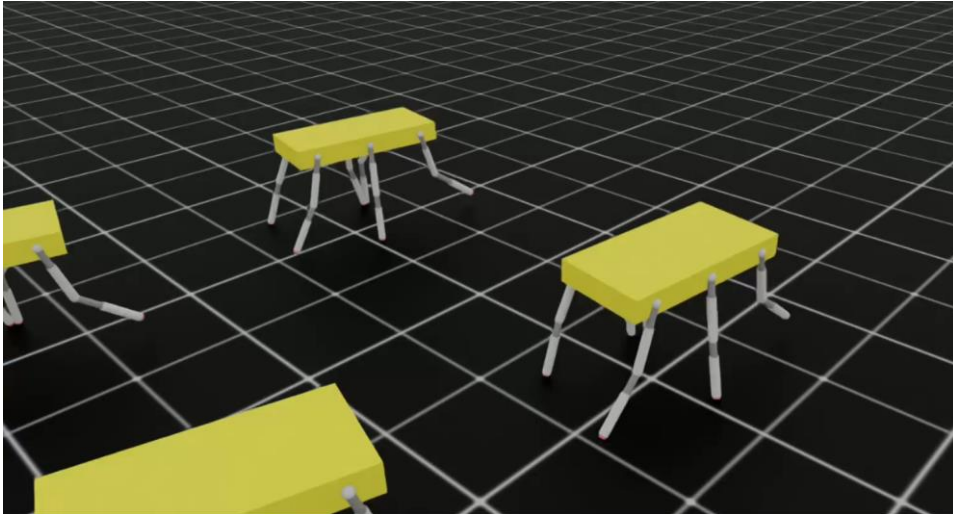
| 종합 비교표(요청 지면 기준)

	수치 (Go2 vs Hugo)	우세	의미
지형 추종 계수	0.97 vs ≈ 0	Hugo	지형 요철을 다리에서 흡수, 몸통 수평 유지
수직 안정 (체고%)	1.78 vs 1.02	Hugo (1.7배)	몸 크기 대비 흔들림이 절반 수준
속도 달성	83% vs 104%	Hugo	유일하게 계획 목표(>0.3m/s) 달성
틸트 RMS	5.10° vs 4.47°	Hugo	기울기 작고, 지형 변화에도 불변
roll RMS	3.88° vs 2.45°	Hugo (1.6배)	좌우 안정 — 6족 정적 지지의 강점
각속도 RMS	0.44 vs 0.36	Hugo (1.2배)	탑재 센서에 유리한 저진동
수직 RMS 절대 [mm]	6.63 vs 10.76	Go2	체고 3.5배 차이의 구조적 결과
pitch RMS	3.28° vs 3.55°	Go2 (근소)	빠른 전진 보행의 앞뒤 요동 비용

P-R 다족 구조는 $\pm 30\text{mm}$ 비정형 지면에서 명령 속도를 완주하며(104%), 몸통이 지형을 타지 않고(추종 계수 ≈ 0) 체고 대비 수직 변동을 4족의 절반 수준(1.02%)으로 유지했다.

성능 지표

| 비교 영상





성능 지표

| 비교 영상



2026.07.28

세종대학교 Medical Robotics LAB.



CPG + DRL 보행 학습



CPG (Central Pattern Generator)

CPG(Central Pattern Generator)는 척수에 존재하는 신경 회로로, 상위 명령 없이도 보행의 주기적 리듬을 자체 생성하는 것을 의미함.

- 제뇌 고양이 실험(Decerebrate Cat)에서 뇌간에 전기 자극만 주어도 걷기 시작하고, 자극 세기를 높이면 걸음이 빨라지다 자동으로 뒹굴음으로 전이 되는 것을 확인 했음.
- 보행 제어를 상위 정책(무엇을 할지)과 하위 리듬(어떻게 걸을지)으로 분리할 수 있다는 생물학적 근거. 현재 우리 시스템은 이 둘을 하나의 신경망이 모두 담당하고 있음.
- CPG는 분산 제어, 중복성 처리 능력, 빠른 제어 루프, 간단한 제어 신호에 의한 움직임이 가능함

CPG (Central Pattern Generator)

CPG(Central Pattern Generator)는 척수에 존재하는 신경 회로로, 상위 명령 없이도 보행의 주기적 리듬을 자체 생성하는 것을 의미함.

왜 정현파가 아니라 비선형 발진기인가

- CPG는 대신 비선형 발진기(Hopf oscillator)를 쓰는데, 어떤 초기 상태에서 출발해도 궤적이 하나의 극한 순환(limit cycle)으로 수렴함.
- 리듬의 안정성과 전환의 연속성을 수학적으로 보장하는 구조로, 걸음새가 파라미터로 표현되므로 상위층에서 지형에 따라 걸음새를 지정하는 설계가 가능해짐.
- 다리 6개 = 발진기 6개
- 위상차 행렬이 곧 걸음새 정의 → tripod / wave / ripple을 값 조합만으로 전환

정현파	비선형 발진기
걸음새 전환 시 불연속	부드럽게 새 리듬으로 이행
피드백 삽입 불가	외란 후 원 궤도로 자동 복귀

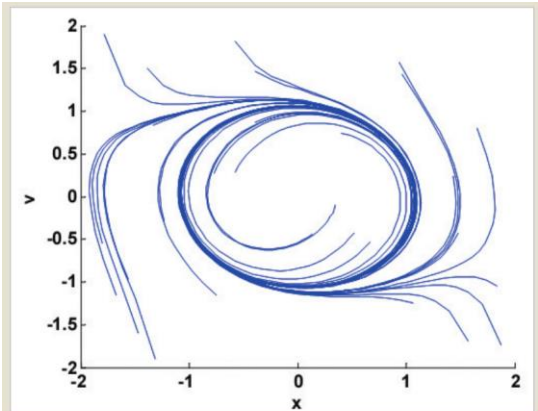
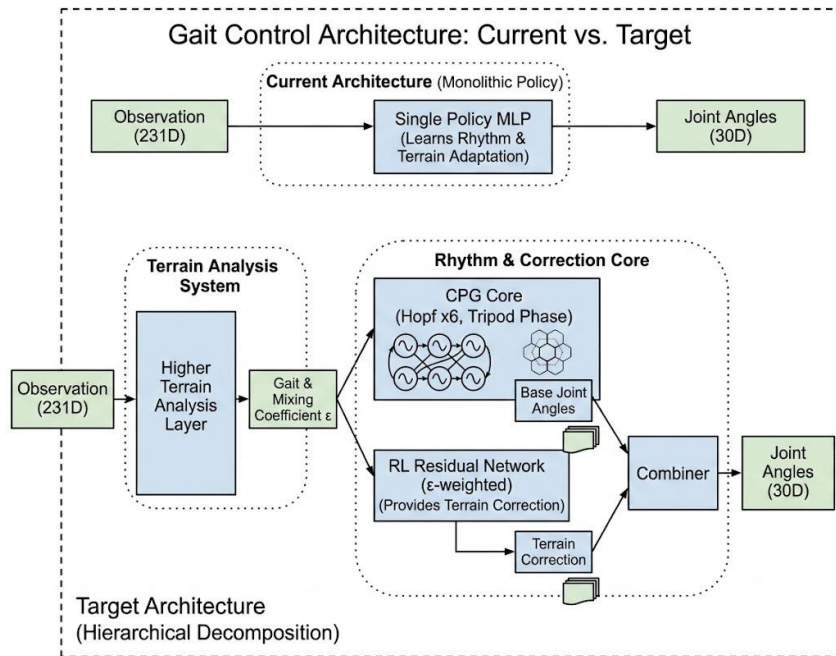


그림 1 임의의 시작 조건에서 시작된 발진 신호들의 제한된 사이클로의 수렴성

CPG (Central Pattern Generator)

CPG 단독은 리듬은 안정적이거나 지형 적응이 불가능하고, DRL 단독은 자유도가 클수록 탐색 공간이 넓어져 학습이 어려움(6족 30차원 액션에서 실제로 겪은 문제). 따라서 CPG가 기본 보행 리듬을 깔고 그 위에 RL이 보정값을 더하는 residual 구조를 적용함.



선행 연구(Huang et al., 2025)는 이 구조로 CPG 단독 대비 한계 지형 성능을 약 2배 향상시켰다(시물 요철 13cm·경사 18°·계단 8cm).
다만 CPG 파라미터와 ϵ 가 고정·수동 튜닝이라는 한계가 있으며, 이는 우리 목표인 free/adaptive gait와 상충하므로 **파라미터 자체를 학습 대상으로 확장하는 것이 우리의 차별점이 됨.**

적용 시 기대효과

탐색 공간 축소: 리듬을 CPG가 담당 → RL은 보정만 학습 (현재 30차원 전량 학습 대비)
걸음새 제어 획득: 위상차 조정으로 tripod/wave 전환 — 현재 정책은 걸음새를 지정할 수 없음

향후 계획

6~7월에 URDF 설계와 강화학습 환경 구축·학습을 완료했으며(현재 지점), 8월부터 CPG 설계와 상위 지형 분석층 분리에 착수함.
9월 최종 정책 산출을 거쳐 10월에 상세 하드웨어 설계 사양을 제공하는 일정임.

타임라인

시기	내용	상태
6월	6족 URDF 설계 · 레퍼런스 조사 · RL 기본 env 설계	완료
7월	RL env 조정 및 학습 · CPG 레퍼런스 조사	완료 (본 발표 결과)
8월	CPG 설계·적용 · 상위 지형 분석층 분리 (관측 차원 축소)	▶ 진행 예정
9월	CPG 수정 · RL env 조정 및 최종 정책 산출	예정
10월	상세 하드웨어 설계 사양 제공	예정